MetroLake

Simulador y plataforma de datos del Metro de Medellín

Mantenimiento predictivo · Clima · Afluencia

Documentación técnica

Código · Servicios y microservicios · Arquitectura · Despliegue

Repositorio	github.com/Ssanchezl3/MetroLake
Rama	feature/frontend-simulador
App desplegada	Cloud Run (us-central1)
Proyecto GCP	terraform-demo-metro
Lenguaje	Python 3 (Flask) + HTML/CSS/JS

1. ¿Qué es MetroLake?

MetroLake es una plataforma que **simula y analiza** los datos operativos del Metro de Medellín. Genera tres tipos de información —**pasajeros** (torniquetes), **vibración** (acelerómetros de la vía) y **clima** (pluviómetros)— y los publica en la nube de Google (GCP) para almacenarlos, consultarlos y evaluar el riesgo operativo.

El proyecto tiene dos grandes bloques:

- **Pipeline de datos:** los simuladores producen eventos y un publicador los envía a Google Cloud Pub/Sub; de ahí se cargan automáticamente a BigQuery.
- **Frontend web (interfaz):** una aplicación Flask desplegada en Cloud Run que permite generar datos, verlos en vivo, consultarlos por día, comparar días y evaluar el riesgo diario.

2. Objetivos funcionales

- **Simular** la operación del metro con datos realistas (incluye la lógica de días de semana vs. fin de semana y horas pico).
- **Trazabilidad:** consultar los datos almacenados por día y filtrarlos por fuente y origen.
- **Comparación:** contrastar dos días (A vs. B) lado a lado.
- **Riesgo diario:** calcular un nivel BAJO / MEDIO / ALTO con criterios explicados.

3. Arquitectura general

La arquitectura sigue un patrón de **capas** sobre GCP. Los simuladores alimentan una capa de velocidad (mensajería) que descarga en un almacén analítico (BigQuery), sobre el cual operan el modelo predictivo y el frontend.

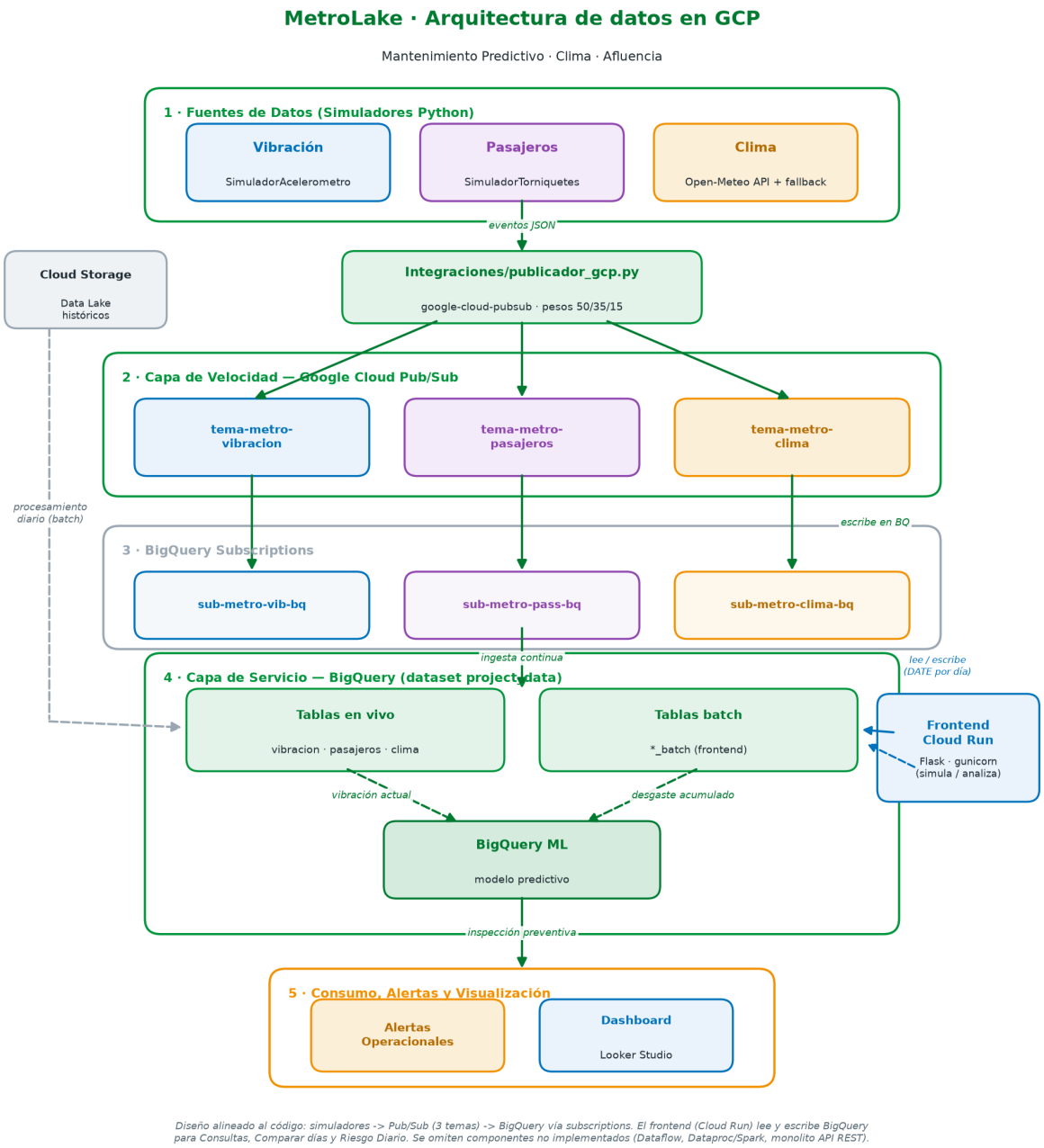


Figura 1. Arquitectura de datos de MetroLake en GCP.

3.1 Flujo de datos, paso a paso

- **1. Fuentes:** tres simuladores en Python generan eventos (vibración, pasajeros, clima). El clima intenta datos reales de la API pública Open-Meteo y, si no hay red, usa un fallback.
- **2. Publicación:** `publicador_gcp.py` envía cada evento al tema de Pub/Sub correspondiente, con una mezcla aproximada de 50% vibración, 35% pasajeros, 15% clima.
- **3. Mensajería (capa de velocidad):** tres temas de Pub/Sub reciben los eventos: `tema-metro-vibracion / -pasajeros / -clima`.
- **4. Carga a BigQuery:** tres *BigQuery subscriptions* (`sub-metro-*-bq`) escriben los mensajes en las tablas.
- **5. Almacén (capa de servicio):** BigQuery (dataset `project_data`) guarda las tablas en vivo y las tablas batch, y alimenta BigQuery ML.
- **6. Consumo:** el frontend (Cloud Run) lee/escribe BigQuery; además hay alertas y un dashboard (Looker Studio) para visualización.

4. Servicios y microservicios usados

4.1 Servicios de Google Cloud (GCP)

Servicio	Rol en MetroLake
Cloud Run	Ejecuta el frontend web (contenedor serverless, escala a cero, HTTPS).
Pub/Sub	Mensajería: 3 temas que reciben los eventos de los simuladores.
BigQuery	Almacén analítico (dataset project_data) con 6 tablas; agrupa por fecha.
BigQuery subscriptions	Cargan automáticamente los mensajes de Pub/Sub a BigQuery.
BigQuery ML	Modelo predictivo (mantenimiento) sobre las tablas.
Cloud Build	Construye el contenedor del frontend al desplegar desde el código.
Artifact Registry	Almacena la imagen del contenedor construida por Cloud Build.
Cloud Storage	Data Lake de históricos del procesamiento batch (opcional).
Looker Studio	Dashboard de visualización (consumo).

4.2 ¿Es monolito o microservicios?

MetroLake es una arquitectura **orientada a servicios desacoplados**, no un único programa monolítico. Cada pieza tiene una responsabilidad clara y se comunica por contratos simples (mensajes JSON en Pub/Sub, tablas en BigQuery, HTTP en el frontend):

- **Simuladores** (librería Python): generan datos. No saben nada de la nube.
- **Publicador** (servicio de ingesta): traduce eventos a mensajes de Pub/Sub.
- **Pub/Sub + subscriptions**: transporte y carga, totalmente gestionados por GCP.
- **Frontend** (microservicio web en Cloud Run): interfaz y consultas, independiente del resto.

Este desacople permite cambiar o escalar una pieza sin tocar las demás (por ejemplo, el frontend se redespliega sin afectar el pipeline de Pub/Sub).

5. Explicación del código

El proyecto se organiza por carpetas según su responsabilidad:

Carpeta / archivo	Qué hace
Fuentes/	Los simuladores: accelerometer.py (vibración), turnstiles.py (pasajeros), weather.py (clima + Op
Ejecucion/	Orquestación: dataset.py (lotes históricos), streaming.py (eventos en vivo), simuladores.py (pun
Persistencia/	serializers.py: exporta a JSON, CSV y NDJSON.
Integraciones/	publicador_gcp.py: publica los eventos a Google Cloud Pub/Sub.
Frontend/	La aplicación web (ver detalle abajo).
docs/	Diagrama de arquitectura y esta documentación.

5.1 Los simuladores (Fuentes/)

- **SimuladorTorniquetes** (pasajeros): calcula entradas/salidas por estación según la hora y el día; reduce la demanda en fin de semana y marca las horas pico.
- **SimuladorAcelerometro** (vibración): genera telemetría RMS por sensor, en función de la fatiga del tramo, la hora, los pasajeros y la lluvia; detecta anomalías.
- **SimuladorClima**: consulta precipitación real en Open-Meteo; si falla, genera datos sintéticos coherentes. Calcula el nivel de alerta (verde/amarilla/naranja/roja).

5.2 El frontend (Frontend/)

Archivo	Responsabilidad
app.py	Servidor Flask y API: genera lotes, emite streaming, guarda, consulta, compara y evalúa riesgo. L
consultas.py	Trazabilidad por día: lista días, carga un día, resume y guarda. Usa BigQuery si está configurado;
bigquery_store.py	Acceso a BigQuery (dataset project_data): lee/escribe las 6 tablas, agrupa por DATE(timestamp) y
riesgo.py	Calcula el nivel de riesgo diario (BAJO/MEDIO/ALTO) con criterios de infraestructura, clima y dem
templates/index.html	Interfaz de una sola página con el menú lateral y las vistas.
static/css, static/js	Estilos (tema Metro de Medellín) y lógica de la interfaz.

5.3 La API del frontend (endpoints principales)

Método y ruta	Para qué sirve
GET /	Carga la interfaz web.
GET /api/estado	Configuración: estaciones, tramos, sensores, pluviómetros.
GET /api/batch	Genera un lote batch de un día de la semana.
GET /api/streaming/evento	Emite un evento de streaming (fuente aleatoria con pesos).
POST /api/guardar/batch	Genera y guarda un lote (en BigQuery o archivo).
GET /api/consultas/dias	Lista los días con datos almacenados.
GET /api/consultas/dia	Tabla y resumen de un día (filtra por origen y fuente).
GET /api/consultas/comparar	Resúmenes de dos días para comparar (A vs. B).

Método y ruta	Para qué sirve
GET /api/riesgo/dia	Nivel de riesgo del día con sus criterios.

6. Datos en BigQuery

El dataset `project_data` tiene seis tablas: una versión **en vivo** y una **batch** por cada fuente. El día se obtiene con `DATE(timestamp)`.

Tabla	Fuente	Campos principales
pasajeros / pasajeros_batch	Torniquetes	event_id, estacion, timestamp, entradas, salidas, total_personas, ocupacion_e
vibracion / vibracion_batch	Acelerómetros	event_id, sensor_id, tramo_id, timestamp, vibracion_rms_mm_s2, fatiga_acum
clima / clima_batch	Pluviómetros	lectura_id, pluviometro_id, timestamp, precipitacion_mm_h, nivel_alerta, esta_

El frontend **lee** estas tablas para Consultas, Comparar días y Riesgo Diario, y puede **escribir** en las tablas `*_batch` los datos que se generan desde la interfaz.

7. Cómo se calcula el Riesgo Diario

El riesgo es la suma de puntos de tres factores; el total se traduce a un nivel (BAJO < 4, MEDIO 4–7, ALTO ≥ 8):

- **Infraestructura (vibración):** porcentaje de anomalías, RMS máximo y fatiga de los sensores.
- **Clima:** precipitación máxima y número de alertas naranja/roja.
- **Demanda (pasajeros):** ocupación estimada y eventos de saturación.

Cada factor explica **por qué** suma puntos, con los datos concretos del día.

8. Cómo ejecutarlo y desplegarlo

8.1 En local

Desde la raíz del proyecto:

```
python -m pip install -r requirements.txt
python Frontend/app.py
# abre http://localhost:8080
```

8.2 En la nube (Cloud Run)

Despliegue del frontend desde el código fuente (crea el contenedor automáticamente):

```
gcloud run deploy metrolake-frontend --source . --region us-central1 \
  --allow-unauthenticated --port 8080 \
  --set-env-vars BQ_DATASET=project_data \
  --set-env-vars GCP_PROJECT_ID=terraform-demo-metro
```

Con la variable `BQ_DATASET` definida, el frontend usa BigQuery (datos permanentes). Sin ella, usa archivos locales. La guía completa está en `DESPLIEGUE_GCP.md`.

9. Glosario rápido

Término	En palabras simples
Pub/Sub	Servicio de mensajería: alguien publica mensajes y otros los reciben.
Subscription	Suscripción que entrega los mensajes de un tema a un destino (aquí, BigQuery).
BigQuery	Base de datos analítica de Google para grandes volúmenes y consultas SQL.
Cloud Run	Ejecuta tu app en un contenedor sin administrar servidores; escala a cero.
Contenedor	Caja con la app y todo lo que necesita para correr igual en cualquier lado.
gunicorn	Servidor que ejecuta la app Flask en producción (más robusto que el de desarrollo).
Serverless	No administras servidores; la nube los enciende/apaga por ti.
Efímero	Temporal: el disco de Cloud Run se borra al reiniciar (por eso se usa BigQuery).